Precept 1: Introduction and Setup, Mostly

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

January 27, 2026

Overview

Welcome to COS 217!

- Preceptor: Polly Ren
  - E-mail: pollyren@princeton.edu
  - Precept: TTh 12:15pm – 1:05pm @ Fine Hall 1001

Get to know each other!

- In groups of 3-4, share:
  - Name
  - Year
  - Major
  - One fun fact (or not-so-fun fact) about yourself
- Designate one person to introduce the group to the class
- You have many resources in COS 217:
  - Lectures
  - ...?

How to get help in COS 217

1. **Lectures**: cover material at a conceptual level
2. **Precepts**: cover material at a lower level, more interactive
   - *Attendance is counted towards your grade!!!*
3. **Ed Discussion**: ask questions about course material or assignments, get help from peers and staff
   - You are encouraged to answer others' questions if you know the answer
   - For assignment help, you must not reveal your design decisions or code
4. **Office Hours**: get one-on-one help from instructors and TAs
   - See also Intro COS Lab Hours
5. **Course Webpage**: find office hours schedule, course info, lecture and precept material, assignment specs, exam info, course policy
   - Assignments 0 and 1 are now available

How to get help in COS 217 (cont.)

6. **Textbooks**: provide more in-depth explanations, examples and exercises
   - *C Programming: A Modern Approach (Second Edition)*, K. N. King
   - *ARM 64-bit Assembly Language*, Larry D. Pyeatt with William Ughetta
   - *The Practice of Programming*, Brian W. Kernighan and Rob Pike
   - *Linux Pocket Guide*, Daniel J. Barrett
7. **Manuals**: refresher on syntax and usage of specific libraries
   - `man` and `tldr` commands in terminal $\rightarrow$ today!
8. **Armlab**: next up

Overview

About assignments in COS 217

- For assignments, you can *write* the code on any computer that has a C development environment
- However. . .
  - Some code-checking tools that you'll need are available only on armlab
  - Assignment submission is done through armlab
  - Assignments are evaluated on armlab
  - Moral: make sure all assignments work on armlab

Your relationship with armlab

- Therefore, you have two main options for working with armlab:
  - Write all your code directly on armlab
    - Pros: everything is in one place
    - Cons: need internet connection to access armlab, can be clunky without an IDE
  - Write code on your own computer, then sync files to armlab for testing and submission
    - Pros: can use your favourite IDE, works offline
    - Cons: need to remember to sync files to armlab
- A third(-ish) option: VSCode with the Remote-SSH extension (my recommendation)
  - Pros: graphical IDE experience, code is saved directly on armlab
  - Cons: still requires internet connection, lose some experience directly interacting with the terminal
  - See appendix slides for setup instructions

What is this armlab thing anyway?

Not this:

- arm is a type of computer architecture
- armlab is a remote server of two computers with arm processors
- Connect to it from your own computer using `ssh` (secure shell)

Connecting to armlab remotely

- `ssh netid@armlab.cs.princeton.edu`
  - Replace `netid` with your Princeton NetID
  - Type your NetID password when prompted (you won't see any characters as you type, but your keystrokes are being recorded)
- See more instructions on *A Minimal COS 217 Computing Environment* handout
  - Once you've connected, configure bash, emacs and splint by following the handout
- To disconnect, use `exit` or `logout`

Optional: How to avoid entering your password every time

- Use SSH keys for passwordless authentication
- Generate an SSH key pair on your local machine using `ssh-keygen`
  - Accept the default file location
  - You can leave the passphrase empty (just press enter twice)
- Copy the public key to armlab using `ssh-copy-id netid@armlab.cs.princeton.edu`
- Next time you connect to armlab, no need to type your password :)
  - But this does not bypass Duo 2FA :(

Optional: How to avoid issuing the full `ssh` command

- On your own computer, create or edit the SSH config file at `~/.ssh/config`
- Add the following lines:

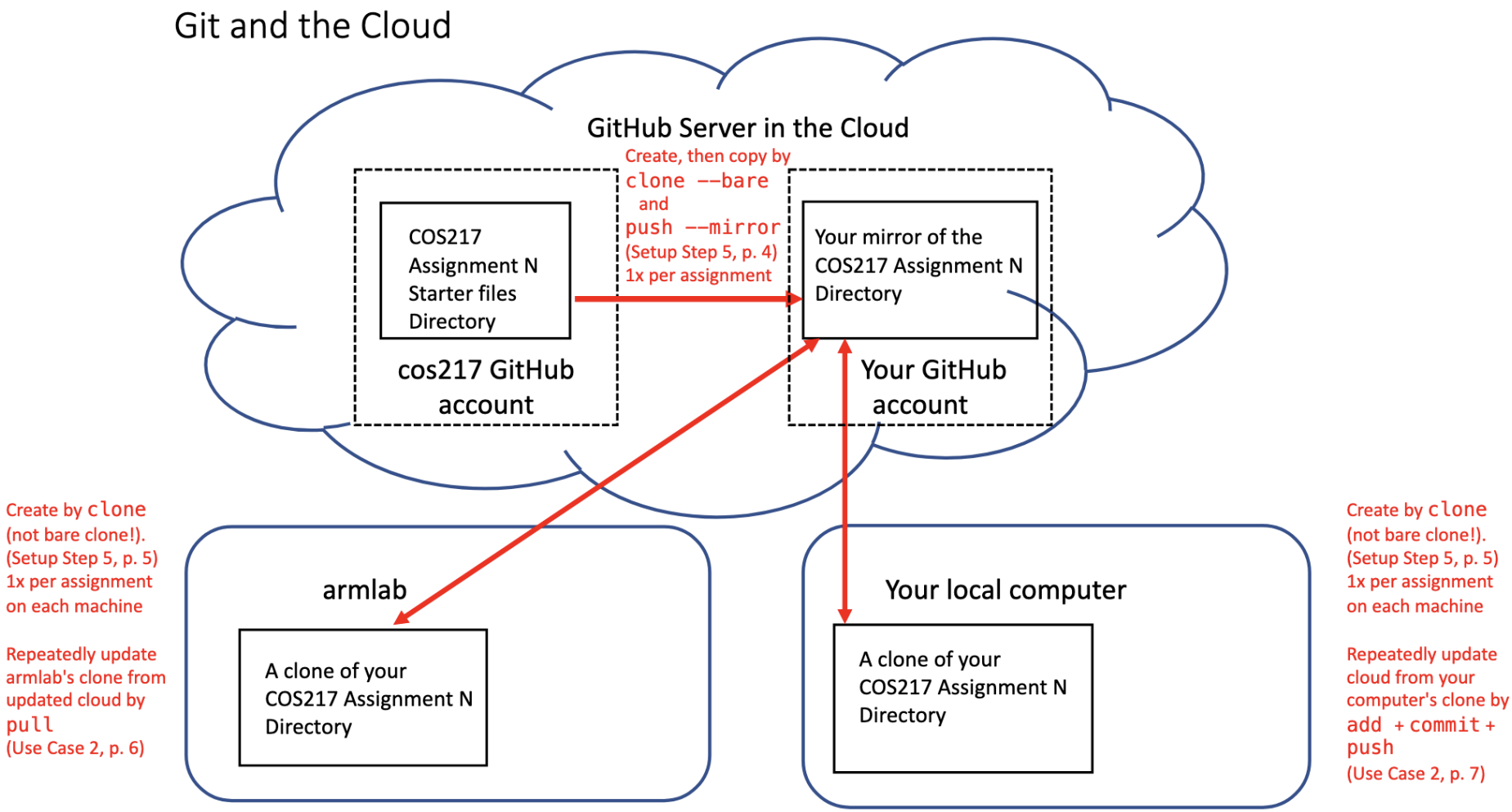
```
Host armlab
    HostName armlab.cs.princeton.edu
    User netid
```

- Replace `netid` with your NetID
- Now you can connect to armlab by simply typing `ssh armlab`

Git and GitHub

- We will the Git version control system to organise our repositories
- We will use GitHub to host our Git repositories remotely
- The starter files, test data and tools for every assignment are in a COS217 repository
- Typical workflow:
  - Copy the files from the COS217 repository to your own GitHub repository
  - Add, commit and push changes to GitHub frequently
  - If developing locally, pull changes from GitHub to armlab before testing and submission
  - You will get practice with this in Assignment 0

Git, GitHub, armlab and you



Details in the *Git and GitHub Primer* handout

Overview

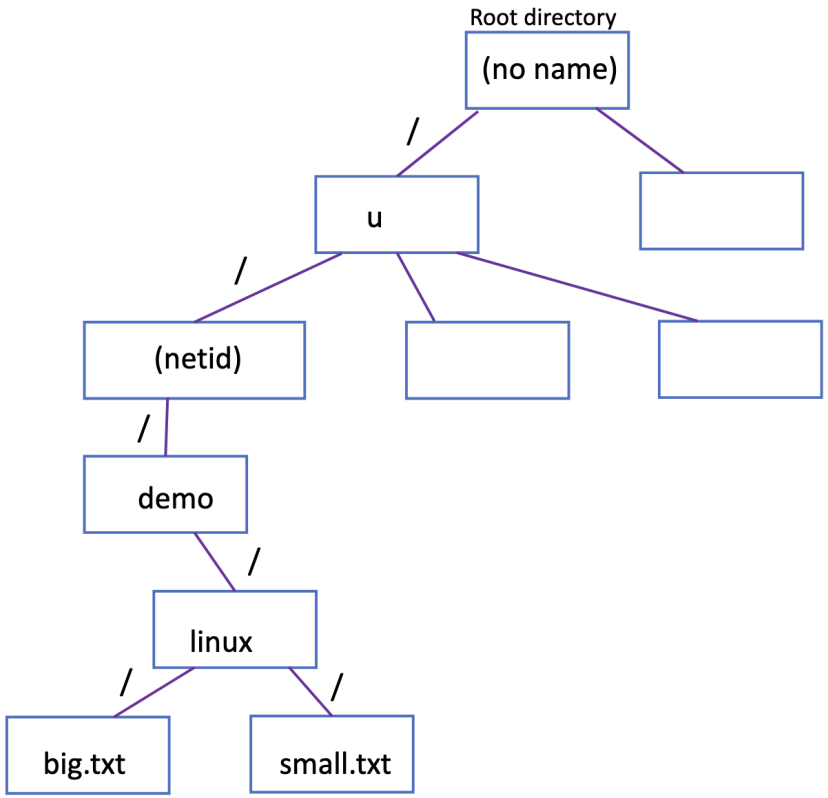
How programs run on a computer: a 10,000 foot view

- Hardware: physical components of a computer (CPU, memory, disk, etc.)
- Operating system (OS): *software* that manages hardware resources and provides services for computer programs
  - Examples: Linux, MS Windows, macOS, UNIX
- System calls: interfaces provided by the OS for programs to request services (e.g., file operations, process management, networking)
  - Examples: `brk()`, `fork()`, `execvp()`, `open()`, `close()`, etc.
- Standard C functions: built on top of system calls to provide higher-level functionality
  - Required by the C standard
  - Examples: `malloc()`, `printf()`, `fopen()`, `fclose()`, etc.
- Applications use these functions to perform tasks

Where does the shell fit in?

- The shell is just another program that runs on top of the OS
  - Can write your own shell (used to be Assignment 7)
- The shell interprets user commands and translates them into system calls to the OS
- An original Unix shell was named the Bourne shell
  - Bash is a successor of the Bourne shell, thus named bash: "Bourne Again Shell"
- There are many other shells (e.g., zsh, csh, fish)

File hierarchy in Linux



- Everything is a file in Linux (even directories!)
- Files are organised in a hierarchical directory structure
- The top-level directory is called the root directory, denoted by a forward slash (/)
- You can navigate the file system using commands like `cd`, `ls`, `pwd`, etc.

Manual pages

- The `man` command displays the manual pages for built-in commands and library functions
  - Example: `man ls` shows the manual page for the `ls` command
  - Use the arrow keys to navigate, and press `q` to quit
  - Sometimes need to specify which section of the manual, e.g., `man 3 printf`
    - Section 1: User commands (`ls`, `cd`, etc.)
    - Section 2: System calls (`open()`, `fork()`, etc.)
    - Section 3: C library functions (`printf()`, `malloc()`, etc.)
    - ...
- Optional: You can also use `tldr` for simplified explanations and examples
  - Example: `tldr ls` shows a brief summary and common usage examples for the `ls` command
  - (You may need to install `tldr` using `pip install tldr` or similar)
- To learn more: `man man` and `tldr tldr`

Commonly-used commands

- Directory-related commands
  - `cd`: change directory
  - `ls`: list files in a directory
  - `pwd`: print working directory
  - `mkdir`: make a new directory
  - `rmdir`: remove an *empty* directory
- File-related commands
  - `cp`: copy files
  - `mv`: move or rename files
  - `rm`: remove files
  - `touch`: create an empty file or update file timestamp
  - `cat`: concatenate and display file contents
  - `more`: view file contents one page at a time
  - `less`: less is more
  - `most`: most is less and more?
- See *The Linux Operating System and the Bash Shell* handout for a more comprehensive list

Data streams and redirection

- Every process has three standard data streams:
  - Standard input (stdin): file descriptor 0
  - Standard output (stdout): file descriptor 1
  - Standard error (stderr): file descriptor 2
- By default, stdin is connected to the keyboard, stdout and stderr are connected to the terminal display
- You can redirect these streams using operators:
  - `>`: redirect stdout to a file (overwrite)
  - `>>`: redirect stdout to a file (append)
  - `2>`: redirect stderr to a file
  - `&>`: redirect both stdout and stderr to a file
  - `<`: redirect stdin from a file
- Can combine them: `./prog < in.txt > out.txt 2> error.log`

Other useful bash features

- Command history: use the up and down arrow keys to navigate through previously entered commands
  - Use `Ctrl + R` to search through command history
  - `history` command to view a list of past commands (this list is a file that you can edit!)
- Tab completion: use Tab to auto-complete file and directory names
- Piping: use the pipe operator | to send the output of one command as input to another command
  - Example: `ls -l | less` views a long list of files one page at a time
- Aborting a process: press `Ctrl + C` to terminate a running command
  - This sends a SIGINT signal to the process
  - Useful for stopping programs that are taking too long or stuck in an infinite loop

Overview

Summary

- COS 217 has many resources to help you succeed
  - Please make use of them!
- You will be using armlab for assignments, so make sure to set up your environment
- Familiarise yourself with Linux and bash commands, as they will be essential for your work in this course
- Dates to remember:
  - Assignment 0: due Tue, 2/3 at 9pm
  - Assignment 1: due Tue, 2/10 at 9pm

Overview

Optional: VSCode with Remote-SSH

- Install VSCode locally (without connecting to armlab)
- Install the Remote-SSH extension in VSCode
- Open the Command Palette (Cmd+Shift+P or Ctrl+Shift+P), type "Remote-SSH: Connect to Host...", and select it
- If you have set up the SSH config file as described earlier, select `armlab` from the list
  - Otherwise, type `netid@armlab.cs.princeton.edu` and press enter
- A new VSCode window should open, connected to armlab
- You can now view and edit files on armlab directly from VSCode

Note: It is easy to get mixed up about whether you are working on your local machine or on armlab. If you are connected via Remote-SSH, the lower left corner of the VSCode window will indicate this. You can also open a terminal in VSCode and use the `hostname` command to check which machine you are on.